

Software Test Plan (STP)

Hostel Management System

Project	Hostel Management System
Version	1.0
Authors	Pranav Shashikiran Atreya (PES2UG24CS912) Kaushik Balaji (PES2UG24CS220) Kanishk Kartik (PES2UG24CS213) Harshith Manish (PES2UG24CS187)
Team Number	12
Date	15-09-2026
Status	Draft for Stage 2 Submission

1. Introduction

Purpose: This Software Test Plan defines the objectives, scope, approach, environment, schedule, responsibilities, risks, metrics, and traceability for testing the Hostel Management System (HMS) version 1.0. It is prepared as the Stage 2 test-planning artifact and uses the SRS requirements baseline as the primary source of test coverage.

Scope: Testing covers authentication and authorization, student management, hostel and room management, room availability, room allocation, transfer, vacation, allocation history, occupancy reporting, non-functional requirements, and the five security requirements defined in the SRS. The project scope is limited to managing student details and hostel room allocation.

The SRS states that implementation shall use one permitted stack: Python with Django or Flask, or PHP with Laravel. The test plan is therefore framework-agnostic at the requirements level; the actual test environment shall match the stack selected for implementation.

1.1 Objectives

- Verify that implemented functions satisfy their SRS acceptance criteria.
- Detect functional, integration, security, usability, robustness, and data-integrity defects before acceptance.
- Demonstrate traceability from every SRS requirement to one or more test cases.
- Provide repeatable entry/exit criteria and measurable reporting for the Stage 2 test process.

1.2 Definitions and Acronyms

Term	Definition
HMS	Hostel Management System
SRS	Software Requirements Specification
STP	Software Test Plan
FR	Functional Requirement
NFR	Non-functional Requirement
RTM	Requirements Traceability Matrix
RBAC	Role-Based Access Control
UAT	User Acceptance Testing
TC	Test Case

2. Test Items

- Authentication and authorization module
- Student management module
- Hostel and room management module
- Room availability and occupancy calculation
- Room allocation, transfer and vacation module
- Allocation history and reporting module
- Cross-cutting non-functional requirements
- Security controls: password hashing, RBAC, session timeout, HTTPS/secure cookies, audit logging

3. Features to be Tested

The following features are directly traceable to the SRS requirement IDs and form the planned functional, non-functional, and security test coverage.

SRS ID(s)	Feature / test focus	Primary test case(s)
HMS-F-001	User authentication with valid/invalid credentials	TC-AUTH-01
HMS-F-002	Role-based authorization for Student/Warden/Administrator	TC-AUTH-02
HMS-F-003	Logout and session invalidation	TC-AUTH-03
HMS-F-004..007	Create, view, update/deactivate, search/filter student records	TC-STU-01..04
HMS-F-008..010	Hostel records, room records, and availability states	TC-ROOM-01..03
HMS-F-011	Block over-capacity room allocation	TC-ROOM-04
HMS-F-012..014	Allocate student, block duplicate active allocation, transfer student	TC-ALLOC-01..03
HMS-F-015..017	Vacate allocation, view current allocation, view allocation history	TC-ALLOC-04..06
HMS-F-018	Occupancy and allocation summary	TC-REPORT-01
HMS-NF-001..007	Performance, availability, integrity, usability, maintainability, robustness, accessibility	TC-NFR-01..07
HMS-SR-001..005	Password hashing, RBAC, timeout, HTTPS/cookies, audit logging	TC-SEC-01..05

4. Features Not to be Tested

- Physical hostel construction, maintenance, or other non-software physical operations.
- Biometric hardware control or specialized hardware interfaces; the SRS explicitly states no specialized hardware is required.
- External institutional ERP integration.
- Payment processing, food/mess management, attendance management, and unrelated hostel services excluded from the SRS scope.
- Internal correctness of third-party browser engines, database engines, or framework internals; the HMS is tested at the application boundary.
- Infrastructure backup administration beyond verifying that the application behaves correctly in the configured environment.

5. Test Approach / Strategy

Test Levels

Level	Purpose	Application to HMS
Unit testing	Verify individual functions/components in isolation.	Validation helpers, models/entities, allocation rules, permission checks, forms/controllers.
Integration testing	Verify interactions between application components and database/session layers.	Student-room-allocation updates, occupancy calculations, authentication + RBAC, audit logging.
System testing	Verify complete end-to-end workflows against SRS acceptance criteria.	Login -> student lookup -> room selection -> allocation -> transfer/vacation -> reporting.
Acceptance / UAT	Confirm the implemented system is acceptable to intended users.	Representative Student, Warden and Administrator workflows and acceptance criteria.

Test Types

- Functional testing - verify each HMS-F requirement and its acceptance criterion.
- Regression testing - rerun affected functional tests after defect fixes or changes.
- Negative testing - invalid credentials, duplicate records, unavailable/full rooms, unauthorized actions, missing records, invalid input.
- Data integrity testing - verify one active allocation per student, room capacity rules, historical allocation preservation, and occupancy counts.

- Performance testing - measure response-time target HMS-NF-001 using a representative project workload.
- Usability and accessibility testing - verify the core workflows, labels, validation messages, keyboard access, and 1280 px layout requirement.
- Security testing - validate HMS-SR-001..005 using controlled test accounts, direct URL checks, session controls, HTTPS configuration, and audit records.

Entry Criteria

- A stable build is available and can be started in the test environment.
- Database schema/migrations are applied and representative test data is available.
- Required user roles (Student, Warden, Administrator) and test accounts are configured.
- The selected application stack and dependencies are installed and the application is reachable.
- Test cases and expected results have been reviewed.

Exit Criteria

- 100% of planned test cases are executed.
- No Critical defects remain open in authentication, authorization, student-data handling, room-capacity enforcement, or allocation integrity.
- All High-priority functional requirements pass their acceptance tests or have a documented approved deviation.
- All five security requirements have been executed and have satisfactory results.
- Non-functional requirements have measurable evidence or documented evaluation results.
- RTM coverage reaches 100% of SRS requirements with final status updated after execution.

5.1 Security Validation

Security validation is performed against the two security objectives and five security requirements defined in Section 5.1 of the SRS.

SRS Security ID	Validation activity	Expected result
HMS-SR-001	Inspect stored user records and verify authentication using known test credentials.	Passwords are stored as one-way hashes; plaintext password values are not present.
HMS-SR-002	Log in as Student and attempt staff-only pages/functions by navigation and direct URL.	Unauthorized access is rejected and no protected staff data is returned.
HMS-SR-003	Create authenticated session, remain inactive for at least configured 30-minute timeout, then request a protected page.	Re-authentication is required after timeout.
HMS-SR-004	Execute authenticated traffic in deployed HTTPS configuration and inspect session/cookie security settings.	HTTPS is used and the authenticated session cookie is marked Secure in deployment.
HMS-SR-005	Perform critical create/update/allocate/transfer/vacate/role actions and inspect audit entries.	Timestamp, user identity, action type and affected record ID are logged; passwords are absent.

6. Test Environment

The exact stack is selected from the permitted technologies in the SRS. The test environment shall mirror the implemented choice.

Item	Planned environment
Client	Current Chrome / Edge / Firefox on Windows or a modern mobile browser where applicable
Application stack	One selected implementation stack: Python + Django, Python + Flask, or PHP + Laravel
Database	Relational database supported by the selected framework/ORM
Network	HTTP for local testing; HTTPS for deployed security validation
Test data	Synthetic student, hostel, room, allocation, role and audit records; no real personal data
Testing tools	Browser developer tools, framework test runner, API client where needed, database query tools, and a spreadsheet/log for execution results
Defect tracking	Project issue log maintained by the team; defects linked to requirement and test-case IDs

7. Test Schedule

The following are the team's planned Stage 2 milestones; they are planning dates rather than institution-assigned deadlines.

Milestone	Schedule	Output
Test case design	15-09-2026 to 16-09-2026	Approved test cases mapped to SRS requirements
Test environment setup	After the first stable project build is available	Ready test environment and test data
Test execution start	After the test environment and required test data are ready	Initial execution results
Functional and integration testing	After each stable build / module completion	Test execution records and defect reports
Regression testing	After defect fixes and before final acceptance	Updated test results confirming corrected functionality
Security validation	After authentication and authorization functionality are implemented	Security validation results
UAT / Acceptance testing	After all High-priority functional requirements pass system testing	UAT results and acceptance status
Final test summary	After completion of test execution and closure/acceptance of critical defects	Final Test Summary Report

8. Test Deliverables

- Software Test Plan (this document).

- Detailed test cases covering all SRS requirements.
- Synthetic test data and test accounts.
- Test execution log/results with Pass/Fail/Blocked status and evidence.
- Defect reports with severity, reproducibility, requirement ID and test-case reference.
- Security validation evidence for HMS-SR-001..005.
- Updated Requirements Traceability Matrix.
- Final Test Summary Report and acceptance/sign-off record.

9. Roles and Responsibilities

Role	Name	Responsibility
QA Lead / Team Lead	Pranav	Lead test planning and coordination, define testing strategy, review functional and integration test cases, maintain SRS-to-test traceability, coordinate defect analysis, and oversee final system and acceptance testing.
	Shashikiran Atreya	
Development Lead / Integration Engineer	Kaushik Balaji	Coordinate application development and module integration, prepare stable builds for testing, perform integration testing, analyze and resolve implementation defects, and support regression testing after fixes.
Test Engineer / Validation Engineer	Kanishk Kartik	Develop and execute functional, boundary, negative, and data-validation test cases, verify student/room/allocation workflows, analyze failed test cases, and perform defect retesting and regression checks.
Security & System Test Engineer	Harshith Manish	Conduct security and authorization validation, test role-based access and session handling, perform system-level and performance-oriented testing, verify error handling and data integrity, and document security/test evidence.

10. Risks and Mitigation

Risk	Impact	Mitigation
Stable build delivered late	High	Prepare tests in parallel with development; request early smoke builds.
Test environment/configuration issues	Medium	Document setup, keep a reproducible local environment, and maintain backup test configuration.
Allocation logic defects causing inconsistent data	High	Use atomic allocation/transfer tests, boundary cases, database integrity checks and regression tests.
Insufficient representative test data	Medium	Prepare synthetic students, rooms at 0%, partial and 100% occupancy, and multiple historical allocations.
Security configuration differs between local and deployed environments	High	Reserve security validation for the deployment configuration and record the exact configuration tested.
Defect fixing changes previously working functions	Medium	Run the affected regression suite after every significant fix.

11. Assumptions & Dependencies

- The implemented application will use one of the technology stacks permitted by the SRS.
- Synthetic test data and test accounts will be available before execution.
- The selected framework and database dependencies can be installed in the test environment.
- The team can enable HTTPS for the deployed security-validation environment.
- System clock, database and application timestamps are sufficiently synchronized for audit-log validation.

- The SRS remains the baseline unless a documented requirements change is approved.

12. Suspension & Resumption Criteria

Suspend testing if:

- The test environment is unavailable for more than four cumulative hours or cannot support reliable execution.
- The build is unstable enough to block more than 30% of the planned test cases.
- Database corruption or severe data inconsistency prevents valid result interpretation.
- A Critical authentication, authorization, or allocation defect makes further dependent testing unsafe or meaningless.

Resume testing if:

- Blocking defects are resolved or an approved stable build is provided.
- The environment and database are restored to a known-good state.
- Required test data/accounts are recreated and the affected test cases are re-baselined.

13. Test Case Management & Traceability

The RTM shall be maintained so that every SRS requirement has at least one referenced test case. Test-case IDs are aligned with the SRS acceptance-test references. Execution status shall be updated from N (Not yet verified) to P (Passed) or A (Accepted/approved) after verification, using the status convention defined in the SRS.

SRS ID	Requirement short	SRS ref	Test case	Status	Comments
HMS-F-001	Authenticate user	SRS 4.x	TC-AUTH-01	N	To be updated after execution
HMS-F-002	Enforce role-based access	SRS 4.x	TC-AUTH-02	N	To be updated after execution
HMS-F-003	Logout user	SRS 4.x	TC-AUTH-03	N	To be updated after execution
HMS-F-004	Create student record	SRS 4.x	TC-STU-01	N	To be updated after execution
HMS-F-005	View own student details	SRS 4.x	TC-STU-02	N	To be updated after execution
HMS-F-006	Update/deactivate student	SRS 4.x	TC-STU-03	N	To be updated after execution
HMS-F-007	Search student records	SRS 4.x	TC-STU-04	N	To be updated after execution
HMS-F-008	Manage hostel records	SRS 4.x	TC-ROOM-01	N	To be updated after execution
HMS-F-009	Manage room records	SRS 4.x	TC-ROOM-02	N	To be updated after execution
HMS-F-010	Show room availability	SRS 4.x	TC-ROOM-03	N	To be updated after execution
HMS-F-011	Prevent over-capacity allocation	SRS 4.x	TC-ROOM-04	N	To be updated after execution
HMS-F-012	Allocate student to room	SRS 4.x	TC-ALLOC-01	N	To be updated after execution
HMS-F-013	Prevent duplicate active allocation	SRS 4.x	TC-ALLOC-02	N	To be updated after execution
HMS-F-014	Transfer student room	SRS 4.x	TC-ALLOC-03	N	To be updated after execution
HMS-F-015	Vacate room allocation	SRS 4.x	TC-ALLOC-04	N	To be updated after execution
HMS-F-016	View current allocation	SRS 4.x	TC-ALLOC-05	N	To be updated after execution
HMS-F-017	View allocation history	SRS 4.x	TC-ALLOC-06	N	To be updated after execution
HMS-F-018	View occupancy summary	SRS 4.x	TC-REPORT-01	N	To be updated after execution

SRS ID	Requirement short	SRS ref	Test case	Status	Comments
HMS-NF-001	Performance target	SRS 5	TC-NFR-01	N	To be updated after execution
HMS-NF-002	Availability target	SRS 5	TC-NFR-02	N	To be updated after execution
HMS-NF-003	Allocation data integrity	SRS 5	TC-NFR-03	N	To be updated after execution
HMS-NF-004	Usable responsive layout	SRS 5	TC-NFR-04	N	To be updated after execution
HMS-NF-005	Maintainable architecture	SRS 5	TC-NFR-05	N	To be updated after execution
HMS-NF-006	Controlled error handling	SRS 5	TC-NFR-06	N	To be updated after execution
HMS-NF-007	Accessible core forms	SRS 5	TC-NFR-07	N	To be updated after execution
HMS-SR-001	Password hashing	SRS 5.1.2	TC-SEC-01	N	To be updated after execution
HMS-SR-002	Role-based authorization	SRS 5.1.2	TC-SEC-02	N	To be updated after execution
HMS-SR-003	Session timeout	SRS 5.1.2	TC-SEC-03	N	To be updated after execution
HMS-SR-004	HTTPS and secure cookies	SRS 5.1.2	TC-SEC-04	N	To be updated after execution
HMS-SR-005	Audit logging	SRS 5.1.2	TC-SEC-05	N	To be updated after execution

14. Test Metrics & Reporting

Metric	Definition / calculation	Reporting
% test cases executed	Executed test cases / planned test cases x 100	Per test cycle and final summary
% pass / fail	Passed or failed executed cases / executed cases x 100	Per test cycle and final summary
Requirement coverage	SRS requirements with at least one test reference / total SRS requirements x 100	Target: 100% before exit
Defect density	Number of valid defects / tested functional area or requirement group	Track by severity and module
Defect aging	Time a defect remains open from logging to closure	Review during daily/periodic triage
Security coverage	Security requirements executed / five security requirements x 100	Target: 100% before acceptance
Regression status	Previously passed impacted tests that still pass after fixes / impacted regression tests	Per regression cycle

Reports: execution dashboard, defect log/triage report, security validation result, updated RTM, and final Test Summary Report.

15. Approvals

Role	Name	Signature	Date
QA Lead / Team Lead	Pranav Shashikiran Atreya	Pranav Shashikiran Atreya	15-09-2026
Development Lead	Kaushik Balaji	Kaushik Balaji	15-09-2026
Test Engineer / Validation Engineer	Kanishk Kartik	Kanishk Kartik	15-09-2026
Security & System Test Engineer	Harshith Manish	Harshith Manish	15-09-2026